Simulateur de circuits analogiques en C++

Rapport de soutenance — Projet de 1re année

Auteurs

Enzo Chen
Aurélie Mordice
Milena Catin de Gouttes Lastouzeilles

Mai 2026

Table des matières

1 Introduction et vision du projet **2**
1.1 Le problème adressé . . . 2
1.2 Principe général de la simulation . . . 2

2 Architecture du projet **2**
2.1 Vue d'ensemble des fichiers . . . 3
2.2 Hiérarchie des types . . . 3
2.3 Modélisation . . . 4

3 Partie utilisateur : construire et connecter un circuit **5**
3.1 Créer des fils et des composants . . . 5
3.2 La surcharge `operator==` : un choix de conception . . . 6
3.3 Génération de la vue graphique . . . 6

4 Les Solveurs **6**
4.1 Solveur naïf (`NaiveSolve`) . . . 6
4.2 Solveur par descente de gradient (`Grad_Desc`) . . . 7
4.2.1 La fonction d'erreur et son gradient . . . 7
4.2.2 Calcul de la dérivée pour la résistance . . . 7
4.2.3 Calcul de la dérivée pour la diode . . . 8
4.2.4 Calcul de la dérivée pour le transistor . . . 8
4.2.5 Stratégie de pas adaptatif . . . 9
4.3 Comparaison des deux solveurs . . . 9
4.4 Analyse de la stagnation . . . 9
4.4.1 Stagnation du solveur naïf . . . 10
4.4.2 Stagnation de la descente de gradient . . . 11

5 Les concepts C++20 : contrats à la compilation **11**

6 Analyse critique des choix de conception **11**
6.1 Ce qui fonctionne bien . . . 11
6.2 Choix discutables . . . 12

7 Ce que nous aurions aimé ajouter **12**
7.1 Solveur de Newton–Raphson . . . 12
7.2 Autres extensions envisagées . . . 13

8 Ce qui ne s'est pas bien passé **13**

9 Guide de reprise du projet **13**
9.1 Prérequis . . . 13
9.2 Ajouter un nouveau composant . . . 13
9.3 Modifier le solveur ou en ajouter un . . . 14
9.4 Points de vigilance pour le repreneur . . . 14

10 Conclusion **14**

1 Introduction et vision du projet

1.1 Le problème adressé

Analyser le comportement d'un circuit électronique analogique demande normalement soit du matériel physique (oscilloscope, alimentation, composants), soit un logiciel commercial lourd (SPICE, LTspice). L'objectif de ce projet est de proposer une bibliothèque C++ légère et extensible permettant de simuler numériquement l'état d'équilibre d'un circuit à partir de ses composants et de leurs connexions.

1.2 Principe général de la simulation

Un circuit électronique en régime continu vérifie la loi des nœuds ou **Première loi de Kirchhoff** : la somme algébrique des courants entrant dans chaque nœud est égale à la somme algébrique des courants sortant de ce nœud. Chaque composant vérifie aussi leur **loi tension-courant** : la relation entre la tension aux bornes d'un composant et le courant qui le traverse.

Notre bibliothèque `crt` modélise ce problème comme un **problème d'optimisation** : on cherche les potentiels $V_1, \ldots, V_n$ sur les fils du circuit qui minimisent l'erreur totale

$$E(V_1, \ldots, V_n) = \sum_{w \in \text{fils libres}} \left(\sum_{p \in \text{pins}(w)} I_p(V_1, \ldots, V_n) \right)^2$$

où $I_p(V_1, \ldots, V_n)$ est le courant calculé par le composant associé au nœud (pin) p en fonction des potentiels des fils. Quand $E = 0$ (ou s'en rapproche le plus possible), la loi des nœuds est vérifiée en tout nœud : le circuit est résolu.

Pourquoi une formulation par optimisation ?

Cette formulation présente une certaine flexible : elle permet d'ajouter n'importe quel composant (diode, transistor...) en définissant simplement sa loi $I(V)$ et sa dérivée dI/dV. C'est un choix extensible, au prix d'une convergence parfois plus lente que les méthodes directes.

2 Architecture du projet

Les circuits électroniques que l'on simule sont composés de :

- Components (composants de la simulation : Résistance, Transistor, Diode)
- Wires (Fils)
- Const_Wires (Fils dont la valeur de potentiel est constante)
- Pins (interface entre un composant et un fils)
- Circuit naïf (Contient des fils et des composants)
- Circuit gradient

2.1 Vue d'ensemble des fichiers

Fichier	Rôle
`crtDefs.hpp`	Types fondamentaux : `Pin`, `Wire_ref`, `Component_ref`
`crtWire.{hpp,cpp}`	Nœuds du circuit (fils libres et fils fixes)
`crtComponent.hpp`	Interface abstraite `Component` et template `Comp<N>`
`crtResistor.{hpp,cpp}`	Modèle résistance linéaire
`crtDiode.{hpp,cpp}`	Modèle diode (équation de Shockley)
`crtTransistor.{hpp,cpp}`	Modèle transistor (équation d'Ebers-Moll)
`crtCircuit.{hpp,cpp}`	Conteneur circuit, calcul de l'erreur totale
`crtCircuitGrad.{hpp,cpp}`	Extension avec calcul du gradient analytique
`crtConcept.hpp`	Concepts C++20 pour la vérification statique de types
`crtSolver.hpp`	Solveurs (descente de gradient, résolution naïve)
`crtUtilities.{hpp,cpp}`	Export GraphViz / visualisation

2.2 Hiérarchie des types

```
Wire_ref : shared_ptr sur un wire.
Pin_ref : shared_ptr sur un pin.
Component_ref : shared_ptr sur un component.

Component (interface abstraite)
  +-- Comp<NB_PINS>   (gère le nombre de pins dans les composants)
        +-- Resistor  (2 pins, loi d'Ohm)
        +-- Diode     (2 pins, équation de Shockley)
        +-- Transistor(3 pins, modèle Ebers-Moll)

Wire (noeud de potentiel libre)
  +-- Const_Wire  (potentiel imposé, non modifiable)

Circuit
  +-- vector<Wire_ref>       -- fils libres
  +-- vector<Wire_ref>       -- fils fixes (const_wires)
  +-- vector<Component_ref>  -- composants

GradCircuit : Circuit (hérite de Circuit)
  +-- compute_grad() -- gradient analytique de l'erreur
```

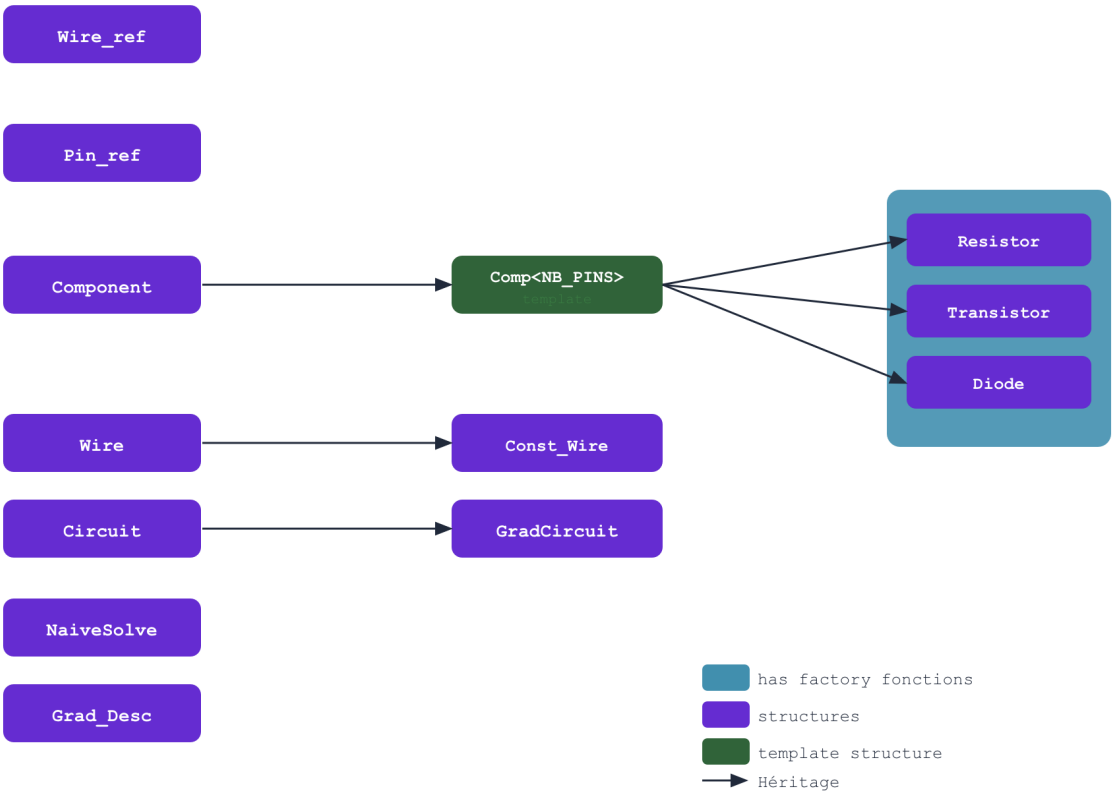


FIGURE 1 – Vision d'ensemble de l'architecture de notre projet.

2.3 Modélisation

La diode est modélisée par le **l'équation de Shockley** :

$$I_D = I_s \left(\exp^{\frac{V_D}{nV_T}} - 1 \right)$$

Avec :

- I_D courant traversant la diode,
- $I_S = 1 \cdot 10^{-12}\ A$ courant de saturation pris à 27 °C,
- V_D la tension aux bornes de la diode,
- $V_T = 25.852\ mV$ le potentiel thermique pris à 27 °C,
- $n = 1$ facteur de qualité idéal.

Nous avons choisi ces paramètres pour la diode afin de représenter le composant à température ambiante. L'utilisateur ne peut pas changer ces caractéristiques car nous avons choisi de ne pas implémenter l'influence de la température sur la diode. Nous avons fait ce choix de conception pour l'efficacité et la simplicité. De même pour le transistor.

Le transistor est modélisé par **l'équation d'Ebers-Moll** :

$$I_E = I_{ES} \left(\exp^{\frac{V_{BE}}{V_T}} - 1 \right), \quad I_C = \alpha_F I_E \quad \text{et} \quad I_B = (1 - \alpha_F)\, I_E$$

Avec :

- I_E le courant émis par le transistor,
- I_C le courant collecteur,
- I_B le courant de base,
- $I_{ES} = 1 \cdot 10^{-12}$ A courant de saturation pris à 27°C,
- V_{BE} la tension aux bornes du transistor entre le point B et le point E,
- $V_T = 25.852$ mV le potentiel thermique pris à 27°C,
- $\alpha_F = 0.98$ le gain de courant .

3 Partie utilisateur : construire et connecter un circuit

3.1 Créer des fils et des composants

L'API est pensée pour être la plus lisible possible. Voici comment construire un circuit simple : une résistance et une diode en série alimentées par 5 V.

```
#include <crt.hpp>

int main()
{
    crt::GradCircuit c;
    auto r{c += crt::make::resistor(10)};
    auto d{c += crt::make::diode()};
    auto [vcc, gnd] = c.make_generator(10);
    auto mid = c.wire();

    vcc == r->pin<0>();
    mid == r->pin<1>();
    mid == d->pin<0>();
    gnd == d->pin<1>();
    solve<Grad_Desc>(c);
    return 0;
}
```

Listing 1 – Exemple d'utilisation : circuit résistance + diode

3.2 La surcharge `operator==` : un choix de conception

La connexion entre un fil et une broche utilise la surcharge de `operator==` :

```
inline void operator==(Wire_ref w, Pin_ref p)
{
    w->pins.push_back(p);  // le fil connait ses broches
    p->wire = w;           // la broche connait son fil
}
```

Listing 2 – Surcharge de == pour la connexion (crtWire.hpp)

Ce choix rend la syntaxe de câblage proche d'un schéma électrique naturel (`fil == broche`). Cependant ce choix peut-être discutable (voir section 6) car `operator==` retourne normalement un booléen.

3.3 Génération de la vue graphique

Le circuit peut être exporté au format GraphViz (DOT) pour être visualisé :

```
#include <crtUtilities.hpp>
#include <fstream>

std::ofstream dot("circuit.dot");
dot << circuit;
dot.close();
crt::utilities::show("circuit.dot"); // genere et ouvre le PDF
```

Listing 3 – Export GraphViz

4 Les Solveurs

4.1 Solveur naïf (`NaiveSolve`)

Le solveur naïf explore chaque fil indépendamment et tente de réduire l'erreur par tâtonnement numérique :

```
for (auto it = c.wires.begin(); it < c.wires.end(); ++it)
{
    auto w = *it;
    double oldV = w->get_potential();

    w->set_potential(oldV + epsilon);
    double err_plus = c.compute_total_error();
    w->set_potential(oldV - epsilon);
    double err_minus = c.compute_total_error();

    w->potential = (err_plus < err_minus)
                 ? oldV + epsilon
                 : oldV - epsilon;
}
```

Listing 4 – Bouble principale du solveur naïf (crtSolver.hpp)

Il évalue deux perturbations symétriques ($V \pm \varepsilon$) et choisit la direction qui réduit l'erreur totale. Simple à implémenter, il souffre de plusieurs limitations :

— Chaque fil est traité séquentiellement, sans coordination globale.
— Le pas $\varepsilon = 10^{-6}$ est fixe : trop grand, il oscille ; trop petit, il converge lentement.
— La complexité est $\mathcal{O}(N_{\text{fils}} \times k_{\text{iter}})$ avec un nombre d'itérations potentiellement très grand.

Une améliorer possible de ce solveur serait de modifier la méthode de sélection des fils : il serait plus judicieux de choisir un fil aléatoirement dans le circuit et de lui ajouter ou retirer ε au lieu de parcourir chaque fil dans un ordre donné.

4.2 Solveur par descente de gradient (`Grad_Desc`)

4.2.1 La fonction d'erreur et son gradient

L'erreur totale est :

$$E(\mathbf{V}) = \sum_{w} \left(\text{err}(w)\right)^2, \quad \text{err}(w) = \sum_{p \in \text{pins}(w)} I_p$$

Son gradient par rapport au potentiel V_i d'un fil i est :

$$\frac{\partial E}{\partial V_i} = 2 \sum_{w} \text{err}(w) \underbrace{\sum_{p \in \text{pins}(w)} \frac{\partial I_p}{\partial V_i}}_{\partial\, \text{err}(w)/\partial V_i}$$

Chaque composant doit donc implémenter la méthode `d(wire_ref, pin_ref)` qui retourne la valeur de $\partial I_p / \partial V_i$ évaluée en les potentiels actuels et nous allons détailler le calcul de ces dérivées dans les parties suivantes.

4.2.2 Calcul de la dérivée pour la résistance

On définit l'expression *pin_in* pour le noeud où l'intensité entre dans le composant (*pin* 0) et l'expression *pin_out* pour le noeud où l'intensité sort du composant (*pin* 1).

Pour une résistance R connectée entre 2 fils de potentiels V_1 et V_2 :

$$I_{\text{pin_in}} = -\frac{V_1 - V_2}{R}, \qquad I_{\text{pin_out}} = +\frac{V_1 - V_2}{R}$$

Ainsi si l'on dérive par rapport à V_1 :

$$\frac{\partial I_{\text{pin_in}}}{\partial V_1} = -\frac{1}{R}, \qquad \frac{\partial I_{\text{pin_out}}}{\partial V_1} = \frac{1}{R}$$

Si l'on dérive par rapport à V_2 :

$$\frac{\partial I_{\text{pin_in}}}{\partial V_2} = \frac{1}{R}, \qquad \frac{\partial I_{\text{pin_out}}}{\partial V_2} = -\frac{1}{R}$$

Sinon, les dérivées sont nulles car les intensités ne dépendent que de V_1 et de V_2.

4.2.3 Calcul de la dérivée pour la diode

La diode suit l'équation de Shockley : $I_D = I_S(e^{U/V_T} - 1)$, avec $U = V_1 - V_2$ et V_1, V_2 les potentiels des fils connectés.

Pour une diode connectée entre les fils de potentiels V_1 et V_2 :

$$I_{\text{pin_in}} = -I_D, \qquad I_{\text{pin_out}} = +I_D$$

Ainsi, les dérivées par rapport à V_1 sont :

$$\frac{\partial I_{\text{pin_in}}}{\partial V_1} = -\frac{I_S}{V_T}e^{U/V_T}, \qquad \frac{\partial I_{\text{pin_out}}}{\partial V_1} = \frac{I_S}{V_T}e^{U/V_T}$$

Les dérivées par rapport à V_2 sont :

$$\frac{\partial I_{\text{pin_in}}}{\partial V_2} = \frac{I_S}{V_T}e^{U/V_T}, \qquad \frac{\partial I_{\text{pin_out}}}{\partial V_2} = -\frac{I_S}{V_T}e^{U/V_T}$$

De même, la dérivée par rapport aux autres potentiels est nulle.

4.2.4 Calcul de la dérivée pour le transistor

Le transistor suit l'équation d'Ebers-Moll, celui-ci est connecté à 3 fils de potentiels respectifs V_B, V_E et V_C :

$$I_E = I_{ES}\left(e^{\frac{V_{BE}}{V_T}} - 1\right)$$

$$I_C = \alpha_F I_E$$

$$I_B = (1 - \alpha_F)\, I_E$$

Avec $V_{BE} = V_B - V_E$

Ainsi, les dérivées par rapport à V_B sont :

$$\frac{\partial I_E}{\partial V_B} = \frac{I_{ES}}{V_T}e^{V_{BE}/V_T}, \qquad \frac{\partial I_B}{\partial V_B} = (1 - \alpha_F)\frac{\partial I_E}{\partial V_B}, \qquad \frac{\partial I_C}{\partial V_B} = \alpha_F\frac{\partial I_E}{\partial V_B}$$

Les dérivées par rapport à V_E sont :

$$\frac{\partial I_E}{\partial V_E} = -\frac{I_{ES}}{V_T}e^{V_{BE}/V_T}, \qquad \frac{\partial I_B}{\partial V_E} = (1 - \alpha_F)\frac{\partial I_E}{\partial V_E}, \qquad \frac{\partial I_C}{\partial V_E} = \alpha_F\frac{\partial I_E}{\partial V_E}$$

Sinon, les dérivées par rapport aux autres potentiels sont nulles car les intensités dépendent uniquement de V_E et de V_B.

4.2.5 Stratégie de pas adaptatif

La descente de gradient naïve avec pas fixe diverge facilement sur des composants non linéaires (l'exponentielle de la diode ou du transistor peut exploser). Le solveur implémente donc un pas adaptatif :

```
double local_step = step;
while (local_step > 1e-15)
{
    // Appliquer le pas candidat
    for (auto w : all_wires)
        if (!w->is_fixed())
            w->set_potential(old_pot[j] - local_step * g[j]);

    double new_error = c.compute_total_error();

    if (new_error < old_error - 1e-15) {
        accepted = true;
        break;              // bonne direction et bon pas
    }

    // Reinitialiser et reduire le pas
    restore_potentials();
    local_step *= 0.95;
}
```

Listing 5 – implémentation du pas adaptatif (crtSolver.hpp)

Le pas est réduit de 5 % à chaque échec jusqu'à trouver un pas acceptable ou descendre sous le seuil 10^{-15}, ce qui interrompt la résolution.

4.3 Comparaison des deux solveurs

Critère	NaiveSolve	Grad_Desc
Nécessite `compute_grad()`	Non	Oui
Gère les non-linéarités	Peu	Bien
Convergence	Lente	Rapide
Robustesse	Moyenne	Moyenne
Extensibilité	Facile	Requiert $\partial I/\partial V$

Quand utiliser quel solveur ?

Le solveur naïf est utile pour vérifier un circuit purement résistif ou pour déboguer un nouveau composant (pas besoin d'implémenter `d()`). La descente de gradient est recommandée pour sa rapidité.

4.4 Analyse de la stagnation

Pour comprendre pourquoi les deux solveurs peinent à converger sur un circuit contenant une diode, on peut visualiser le graphe de l'erreur quadratique sur un circuit contenant 2 résistances et une diode en série (figure 2).



FIGURE 2 – Vue du graphe de l'erreur quadratique avec une ligne horizontale représentant le zéro.

Cette courbe suit l'équation suivante :

$$E(V_3, V_4) = \left(\frac{E_g - V_3}{R_1} - I_s\left(e^{\frac{V_3 - V_4}{V_T}} - 1\right)\right)^2 + \left(I_s\left(e^{\frac{V_3 - V_4}{V_T}} - 1\right) - \frac{V_4}{R_2}\right)^2$$

Avec :

- $E_g = 10V = V_1 - V_2$ tension du générateur avec V_2 potentiel de la masse,
- $R_1 = 10\ \Omega$,
- $R_2 = 5\ \Omega$,
- $I_s = 10^{-12}$ A,
- $V_T = 0{,}02585$ V.

4.4.1 Stagnation du solveur naïf

Le solveur naïf teste, indépendamment pour chaque potentiel, deux perturbations symétriques $V_i \pm \varepsilon$ et choisit la direction qui réduit E. Dans la zone plate de la vallée (figure 2), les deux erreurs vérifient :

$$E(V_i + \varepsilon) \approx E(V_i - \varepsilon) \quad \text{avec} \quad i \in \{3, 4\}$$

La différence est inférieure à la précision numérique du `double`. Le solveur ne peut plus discriminer de direction préférentielle et oscille sans progresser. En termes géométriques : **le gradient numérique d'ordre 1 ne permet plus de distinguer une direction de descente significative**.

4.4.2 Stagnation de la descente de gradient

La descente de gradient est bloquée par la **Zone plate : critère d'arrêt prématuré et échec du pas adaptatif.** Dans la vallée large (figure 2), l'erreur quadratique décroît très lentement. Elle peut atteindre le seuil de tolérance 10^{-12} *avant* d'avoir atteint le fond, car la surface de l'erreur est si peu courbée que le gradient est numériquement indiscernable de zéro loin du minimum réel. Le solveur est géométriquement « coincé » au bord du creux sans pouvoir y entrer.

5 Les concepts C++20 : contrats à la compilation

Un des points techniques les plus soignés du projet est l'utilisation des **concepts C++20** pour vérifier statiquement qu'un circuit passé à un solveur possède toutes les méthodes requises.

```
template <typename Circuit>
concept Grad_Desc =
    NaiveSolve_Circuit<Circuit> &&   // herite du concept naif
    requires(Circuit c) {
        { c.compute_grad() } -> std::ranges::range;
        { *(c.compute_grad().begin()) } -> std::convertible_to
           <double>;
    };
```

Listing 6 – Concept Grad_Desc (crtConcept.hpp)

Si l'utilisateur passe un `Circuit` (sans `compute_grad()`) au solveur par gradient, le compilateur émet un message d'erreur clair :

```
error: crt::Concept::Grad_Desc is not satisfied.
       use crt::GradCircuit or another class with methode compute_grad().
```

Cela évite des erreurs de linkage cryptiques et documente l'API par des contraintes vérifiables.

6 Analyse critique des choix de conception

6.1 Ce qui fonctionne bien

Points forts de l'architecture

- **Extensibilité** : ajouter un nouveau composant ne nécessite que d'hériter de `Comp<N>` et d'implémenter `compute_i()`, `d()`, `name()` et `symbol()`.
- **Séparation solveur/circuit** : les solveurs sont des structs indépendants qui opèrent sur n'importe quel circuit vérifiant le concept approprié. On peut changer de solveur sans toucher au circuit.
- **Concepts C++20** : les erreurs de typage sont détectées à la compilation avec des messages compréhensibles.
- **Polymorphisme par `shared_ptr`** : la gestion mémoire est automatique, sans risque de fuite ou de double libération.

6.2 Choix discutables

Points d'amélioration de conception

- **Surcharge de `operator==` pour la connexion** : retourner `void` sur un opérateur attendu booléen est surprenant et peut dérouter un lecteur. Une méthode `connect(wire, pin)` serait plus explicite et idiomatique.
- **Attribut `potential` public dans `Wire`** : le potentiel est directement accessible en écriture (`w->potential = ...`) dans `NaiveSolve`, contournant ainsi l'encapsulation de `set_potential()`. Un accesseur uniforme serait préférable.
- **Gradient calculé sur les fils fixes aussi** : `GradCircuit::compute_grad()` calcule le gradient pour `all_wires` (libres et fixes), mais le solveur n'applique les mises à jour qu'aux fils libres. Le code est correct mais légèrement redondant en calcul.
- **Pas de tests unitaires** : l'absence de cas de test formels (valeurs attendues connues analytiquement) rend difficile la détection de régressions lors de modifications.
- **Pas d'influence thermique** : La diode et le transistor possèdent des caractéristiques qui dépendent de la température. Or, nous n'avons pas pris cela en compte.

7 Ce que nous aurions aimé ajouter

7.1 Solveur de Newton–Raphson

La méthode de Newton–Raphson est l'algorithme standard des simulateurs de circuits. Elle converge quadratiquement alors que la descente de gradient converge linéairement.

Le système à résoudre est $\mathbf{f}(\mathbf{V}) = \mathbf{0}$, où $f_w = \sum_p I_p$ est l'équation de nœud pour le fil w. La méthode de Newton itère :

$$\mathbf{J} \cdot \Delta\mathbf{V} = -\mathbf{f}(\mathbf{V}^{(k)}), \qquad \mathbf{V}^{(k+1)} = \mathbf{V}^{(k)} + \Delta\mathbf{V}$$

où $J_{ij} = \partial f_i / \partial V_j$ est la **matrice jacobienne**, qui regroupe exactement les dérivées `d(wire, pin)` déjà calculées dans le projet. L'infrastructure est donc presque complète pour implémenter Newton : il manque uniquement l'assemblage de $\mathbf{J}$ et la résolution du système linéaire $\mathbf{J}\Delta\mathbf{V} = -\mathbf{f}$ (par exemple via Eigen ou une décomposition LU maison).

```
// Assembler le Jacobien J et le vecteur f
for (size_t i = 0; i < free_wires.size(); ++i)
    for (size_t j = 0; j < all_wires.size(); ++j)
        for (auto p : all_wires[i]->pins)
            J[i][j] += p->component->d(all_wires[j], p);

// Resoudre J * dV = -f  (LU ou conjugate gradient)
auto dV = linear_solve(J, -f);

// Mise a jour des potentiels
```

```
for (size_t i = 0; i < free_wires.size(); ++i)
    free_wires[i]->set_potential(V[i] + dV[i]);
```

Listing 7 – Pseudo-code Newton-Raphson (non implementé)

7.2 Autres extensions envisagées

— **Simulation en régime transitoire** : introduire une dépendance temporelle et modéliser condensateurs et bobines.
— **Interface utilisateur** : remplacer la sortie DOT/GraphViz par une représentation schématique standard.
— **Tests unitaires** : valider chaque composant sur des circuits dont la solution analytique est connue (diviseur de tension, redresseur idéal, etc.).
— **Influence thermique** : rajouter l'influence de la température dans les caractéristiques de la diode et du transistor.

8 Ce qui ne s'est pas bien passé

— **Convergence instable de la diode** : l'exponentielle de l'équation de Shockley peut atteindre des valeurs astronomiques pour de grandes tensions. Plusieurs tentatives de clamping ($U_{\max} = 0.5\,\mathrm{V}$) ont été essayées puis abandonnées car elles introduisaient des discontinuités dans le gradient. Le backtracking atténue ce problème sans le résoudre fondamentalement.
— **Gestion des cas dégénérés** : un circuit mal câblé (composant connecté à aucun fil) ne génère pas d'erreur explicite mais produit des résultats silencieusement incorrects.
— Nos 2 solvers ne parviennent pas à converger vers une solution en cas de `"plateau"` dans la fonction de l'erreur quadratique.

9 Guide de reprise du projet

9.1 Prérequis

— Compilateur C++ avec support C++20 (`g++ >= 12` ou `clang++ >= 14`).
— Option de compilation : `-std=c++20 -I<chemin_vers_headers>`.
— Optionnel : `neato` (paquet `graphviz`) pour la visualisation.

9.2 Ajouter un nouveau composant

1. Créer `crtMonComposant.hpp` en héritant de `Comp<N>` avec le bon nombre de pin.
2. Implémenter `compute_i()` : calculer les courants à partir des potentiels des fils connectés.
3. Implémenter `d(Wire_ref w, Pin_ref p)` : retourner $\partial I_p / \partial V_w$.
4. Ajouter une fonction factory dans `namespace crt::make`.
5. Inclure le header dans `crt.hpp`.

9.3 Modifier le solveur ou en ajouter un

Un solveur est une struct avec une méthode template `solve(Circuit& c)`. Il doit respecter un concept existant (`NaiveSolve_Circuit` ou `Grad_Desc`) ou encore un nouveau concept. Il peut être ajouté dans `crtSolver.hpp` ou dans un fichier séparé.

9.4 Points de vigilance pour le repreneur

À lire avant de modifier le code

- Le lien entre fil et composant est bidirectionnel : `Wire::pins` pointe vers les broches, et `Pin::wire` pointe vers le fil. Les deux doivent être mis à jour simultanément (c'est ce que fait `operator==`).
- `Const_Wire::set_potential()` lève une exception. Ne pas itérer sur tous les fils sans tester `is_fixed()` avant toute écriture.
- L'ordre d'itération sur les fils dans le gradient doit être cohérent entre `compute_grad()` et le solveur (même ordre `all_wires`).
- `compute_i()` doit être appelé avant `compute_error()` ou `compute_grad()` pour garantir que les courants sont à jour.
- Un nouveau composant ne calculant pas sa dérivée peut être ajouté dans un circuit `naïf` sans erreur à la compilation.

10 Conclusion

Ce projet a permis de construire de bout en bout une bibliothèque de simulation de circuits analogiques en C++ moderne. Les principaux acquis sont :

- La modélisation d'un problème physique (loi des nœuds) comme un problème d'optimisation différentiable.
- L'utilisation avancée du C++20 : concepts, templates, polymorphisme par héritage et concept.
- L'implémentation d'algorithmes d'optimisation (descente de gradient) sur un problème réel non linéaire.
- Une architecture extensible où ajouter un composant n'impacte pas le reste du code.

La prochaine étape naturelle est l'implémentation de la méthode de Newton–Raphson, pour laquelle toute l'infrastructure (calcul des dérivées par composant) est déjà en place. Ce serait l'occasion de valider quantitativement les résultats sur des circuits de référence dont la solution analytique est connue.